```
import cv2
import easyocr
import pandas as pd
import serial
import time
from datetime import datetime

# ============================
# CONFIGURATION
# ============================
REGISTERED_CSV_PATH = r"D:\Authorized_Vehicles.csv"
VEHICLE_LOG_PATH = r"D:\Vehicle_Log.xlsx"
ENTRY_TYPE = "Entry"
DUPLICATE_DELAY = 10
CONFIRM_FRAMES = 3
LED_TIMEOUT = 5
SERIAL_PORT = "COM5"
BAUD_RATE = 9600

# ============================
# SERIAL CONNECTION
# ============================
ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
time.sleep(2)

def send_cmd(cmd):
    """Reliable command sending to Arduino"""
    try:
        ser.write(cmd)
        ser.flush()
        time.sleep(0.1)   # ensures Servo gets command
```

```
    except:
        pass

# ==============================
# CAMERA SELECTION
# ==============================
def get_working_camera():
    for i in range(5):
        cap = cv2.VideoCapture(i, cv2.CAP_DSHOW)
        if cap.isOpened():
            return i
    exit()

camera = get_working_camera()
cap = cv2.VideoCapture(camera, cv2.CAP_DSHOW)

# ==============================
# LOAD REGISTERED DATABASE
# ==============================
registered_df = pd.read_csv(REGISTERED_CSV_PATH)
registered_df["License Plate"] = registered_df["License Plate"].astype(str).str.replace(" ",
"").str.upper()
REGISTERED_DICT = dict(zip(registered_df["License Plate"], registered_df["Faculty ID"]))
REGISTERED_PLATES = set(REGISTERED_DICT.keys())

# ==============================
# OCR READER
# ==============================
reader = easyocr.Reader(['en'])

# ==============================
```

```
# VEHICLE LOG FUNCTION
# ============================
def log_vehicle(plate, status, faculty_id):
    now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    date, time_val = now.split(" ")
    entry = pd.DataFrame([[plate, date, time_val, ENTRY_TYPE, status, faculty_id]],
                         columns=["Vehicle Number","Date","Time","Entry/Exit","Status","Faculty ID"])
    try:
        existing = pd.read_excel(VEHICLE_LOG_PATH)
        final = pd.concat([existing, entry], ignore_index=True)
    except:
        final = entry
    final.to_excel(VEHICLE_LOG_PATH, index=False)

# ============================
# MAIN SYSTEM LOOP
# ============================
last_seen = {}
plate_buffer = {}
last_action_time = time.time()

print("\n🚚 SYSTEM STARTED — Awaiting Vehicles...\nPress 'q' to Stop.\n")

while True:
    ret, frame = cap.read()
    if not ret:
        continue

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    results = reader.readtext(gray)
    now = datetime.now()
```

```
detected = []

for (bbox, text, prob) in results:
    clean = "".join(ch for ch in text if ch.isalnum()).upper()
    if prob > 0.6 and len(clean) >= 6:
        detected.append(clean)

for plate in detected:

    plate_buffer[plate] = plate_buffer.get(plate, 0) + 1

    if plate_buffer[plate] < CONFIRM_FRAMES:
        continue

    plate_buffer[plate] = 0

    if plate in last_seen and (now - last_seen[plate]).total_seconds() < DUPLICATE_DELAY:
        continue

    last_seen[plate] = now
    last_action_time = time.time()

    if plate in REGISTERED_PLATES:
        faculty = REGISTERED_DICT[plate]
        send_cmd(b'G')     # GREEN LED + SERVO OPEN
        log_vehicle(plate, "Registered", faculty)
        print(f"\n✅ REGISTERED: {plate} | Faculty ID: {faculty}")
        print("GATE OPENING... ✅\n")

    else:
        send_cmd(b'R')     # RED LED + BUZZER + SERVO CLOSED
```

```
            log_vehicle(plate, "Unregistered", "N/A")
            print(f"\n❌ UNREGISTERED: {plate}")
            print("Not Allowed. GATE REMAINS CLOSED. 🚫\n")

    # AUTO TURN OFF
    if time.time() - last_action_time > LED_TIMEOUT:
        send_cmd(b'O')

    cv2.imshow("Smart Vehicle Access System", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        send_cmd(b'O')
        ser.close()
        break

cap.release()
cv2.destroyAllWindows()
```